BioBatch Sentinel

A tool-using RL environment for code repair in synthetic bio-monitoring tasks

- Five tasks across two difficulty levels
- Dockerized execution and programmatic verification
- Repeatable episodes with structured logs

The problem

Can an agent inspect, edit, test, and finish small code tasks?

- Agents need tools, state, feedback, and measurable outcomes.
- A benchmark should reward correct behavior, not confident prose.
- BioBatch Sentinel makes the outcome testable.

Design principles

Small, deterministic, observable

- Five focused tasks: two Easy and three Medium.
- Seeded reset and persistent per-episode workspace.
- Programmatic verifier, explicit termination, and reward.
- Oracle and no-op gate before model evaluation.

The five tasks

Synthetic tasks inspired by sensor/process monitoring

- Easy: sensor_calibration — subtract offset, then scale.
- Easy: ph_alarm — strict boundary behavior.
- Medium: do_trend — average valid values; ignore None.
- Medium: contamination_signal — inclusive threshold.
- Medium: recovery_action — ordered priority rules.

Architecture

Separate orchestration from untrusted code execution

- Harness: episodes, model calls, tools, verifier, reward, logs.
- Sandbox control: reset, workspace, tool routing, verification.
- Executor worker: runs Python snippets in a separate container.
- Executor uses network_mode: none and a shared queue volume.

Episode lifecycle

From instruction to measured outcome

- Reset deterministic task state and workspace.
- Provide task instructions and tool schemas.
- Parse actions, execute tools, return results.
- Finish or reach a step/time cap; run verifier and reward.
- Record termination and evidence in JSONL.

Verification gate

Oracle should pass; no-op should fail

- Gate threshold: oracle ≥ 0.95; no-op ≤ 0.05.
- All five oracle scores: 1.00.
- All five no-op scores: 0.00.
- Recorded gate result: PASS.

Evaluation results

Recorded full run: all five tasks solved

- Run completed; model was called.
- 5/5 tasks succeeded; mean reward = 1.00.
- Each task succeeded in five steps.
- Separate mock/idle run failed in one step with reward 0.00.

Observability

Evidence is part of the environment

- JSONL includes model inputs/output, actions, tool results, errors, state hash, reward, tokens, latency.
- Episode-end records termination and elapsed time.
- Evidence includes gate, five task logs, and success/failure examples.
- One-command Docker Compose launch was tested.

Limitations & hardening

Good engineering includes honest boundaries

- Tasks are synthetic, not real-world process-control validation.
- Network isolation is defense-in-depth, not a formal security proof.
- Review workspace permissions and malformed-action handling.
- Repeat tests and scan logs/artifacts for secrets.

Takeaways

A benchmark is a system, not just a prompt

- Clear tasks and schemas make behavior measurable.
- Oracle/no-op baselines detect broken or trivial tasks.
- Isolation, reproducibility, logs, and failure cases build trust.
- Next: clean-clone test, final review, archive, private repository.